

AI-Assisted Coding Lab Assignment=13.3

Name: Pothunuri Akshitha

Roll no: 2403A510C2

Batch no: 05

CSE 2nd year

Task Description #1 – Remove Repetition

Task: Provide AI with the following redundant code and ask it to refactor

Python Code

```
def calculate_area(shape, x, y=0):  
    if shape == "rectangle":  
        return x * y  
    elif shape == "square":  
        return x * x  
    elif shape == "circle":  
        return 3.14 * x * x
```

Prompt:

Refactor the following redundant Python code to make it cleaner, modular, and easier to maintain:

Code:

```
def calculate_area(shape, x, y=0):  
    if shape == "rectangle":  
        return x * y  
    elif shape == "square":  
        return x * x  
    elif shape == "circle":  
        return 3.14 * x * x
```

Requirements:

- *Use either:*
 - *Dictionary-based dispatch, or*
 - *Separate functions for each shape with a dispatcher.*
- *The refactored version should:*
 - *Be modular and scalable (easy to add new shapes).*
 - *Include error handling for unsupported shapes.*
 - *Be clean, readable, and efficient.*

CODE GENERATED:

```
13.3.task1.py > ...
Tabnine | Edit | Test | Explain | Document | Qodo Gen: Options | Test this function
1 def area_rectangle(x, y):
2     return x * y
3
Tabnine | Edit | Test | Explain | Document | Qodo Gen: Options | Test this function
4 def area_square(x, y=0):
5     return x * x
6
Tabnine | Edit | Test | Explain | Document | Qodo Gen: Options | Test this function
7 def area_circle(x, y=0):
8     return 3.14 * x * x
9
Tabnine | Edit | Test | Explain | Document | Qodo Gen: Options | Test this function
10 def calculate_area(shape, x, y=0):
11     shape_dispatch = {
12         "rectangle": area_rectangle,
13         "square": area_square,
14         "circle": area_circle
15     }
16
17     if shape not in shape_dispatch:
18         raise ValueError(f"Unsupported shape: '{shape}'")
19
20     return shape_dispatch[shape](x, y)
21
22 # Example usage:
23 print(calculate_area("rectangle", 5, 10)) # 50
24 print(calculate_area("square", 4))        # 16
25 print(calculate_area("circle", 3))        # 28.26
26
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS CODE REFERENCE LOG
PS C:\Users\HP\Desktop\AI-ASSISTED CODING> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/HP/Desktop/AI-ASSISTED CODING/13.3.task1.py"
50
16
28.259999999999998
PS C:\Users\HP\Desktop\AI-ASSISTED CODING>
```

Observation:

- The code uses *separate functions* for each shape's area calculation (*rectangle*, *square*, *circle*).
- A *dictionary-based dispatcher* maps shape names to their corresponding functions.

- The main function `calculate_area` delegates the calculation via this dispatcher.
- **Default parameter** `y=0` ensures compatibility across shape functions.
- The code includes **error handling** for unsupported shapes using `ValueError`.
- This design is **modular**, making it easy to add new shapes.
- Improves **readability** by separating logic for each shape.
- Encourages **scalability** and maintainability in real-world applications.

Task Description #2 – Error Handling in Legacy Code

Task: Legacy function without proper error handling

Python Code

```
def read_file(filename):
    f = open(filename, "r")
    data = f.read()
```

```
f.close()
return data
```

Prompt:

Refactor the following legacy Python function, which lacks proper error handling for file operations:

Code:

```
def read_file(filename):
    f = open(filename, "r")
    data = f.read()
    f.close()
    return data
```

Task:

Improve this function by adding:

- Proper **error handling** for cases like:
 - File not found
 - Permission issues
 - Other I/O errors
- Use of **context manager** (`with` statement) for safe file handling.
- Ensure the function is **robust, clean, and maintainable**.

CODE GENERATED:

```
Welcome 13.3.task1.py 13.3.task2.py X
13.3.task2.py > ...
Tabnine | Edit | Test | Explain | Document | Qodo Gen: Options | Test this function
1 def read_file(filename):
2     try:
3         with open(filename, "r") as f:
4             data = f.read()
5             return data
6     except FileNotFoundError:
7         return f"Error: The file '{filename}' was not found."
8     except PermissionError:
9         return f"Error: Permission denied to read the file '{filename}'."
10    except IOError as e:
11        return f"An I/O error occurred while reading '{filename}': {e}"
12
13 # Example usage
14 # This will attempt to read 'example.txt' in the same directory
15 output = read_file("example.txt")
16 print(output)
17
```

Output:

```
PS C:\Users\HP\Desktop\AI-ASSISTED CODING> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/HP/Desktop/AI-ASSISTED CODING/13.3.task2.py"
Error: The file 'example.txt' was not found.
PS C:\Users\HP\Desktop\AI-ASSISTED CODING> |
```

Observation:

1. Uses a **context manager (with statement)** to ensure the file is properly closed after reading.
2. Implements **specific error handling** for `FileNotFoundError`, `PermissionError`, and general `IOError`.
3. Returns **informative error messages** instead of raising exceptions or crashing.
4. Keeps the function **clean, readable, and easy to maintain**.
5. Avoids the need for manual file closing, preventing resource leaks.
6. The function handles all common file I/O issues gracefully.
7. Allows calling code to handle errors by returning messages rather than halting execution.
8. Includes an example usage that runs without user input for demonstration.

Task Description #3 – Complex Refactoring

Task: Provide this legacy class to AI for readability and modularity improvements:

Python Code

class Student:

def __init__(self, n, a, m1, m2, m3):

self.n = n

self.a = a

self.m1 = m1

self.m2 = m2

self.m3 = m3

```
def details(self):  
    print("Name:", self.n, "Age:", self.a)  
def total(self):  
    return self.m1+self.m2+self.m3
```

Prompt:

Refactor the following legacy Python class `Student` to improve readability and modularity.

Code:

```
class Student:  
    def __init__(self, n, a, m1, m2, m3):  
        self.n = n  
        self.a = a  
        self.m1 = m1  
        self.m2 = m2  
        self.m3 = m3  
  
    def details(self):  
        print("Name:", self.n, "Age:", self.a)  
  
    def total(self):  
        return self.m1 + self.m2 + self.m3
```

Requirements:

- Use **meaningful variable names** like `name`, `age`, and `marks`.
- Store marks in a **list** or other iterable and use `sum()` to calculate total marks.
- Add **docstrings** for the class and methods.
- Improve the `details` method for **cleaner, more readable output** (use formatted strings).
- Ensure the class is **modular, clean, and easy to maintain**.

CODE GENERATED:

```
Welcome 13.3.task1.py 13.3.task2.py 13.3.task3.py X
13.3.task3.py > ...
Qodo Gen: Options | Test this class
1 class Student:
2     """
3     Represents a student with name, age, and a list of marks.
4     """
5
6     Tabnine | Edit | Test | Explain | Document | Qodo Gen: Options | Test this method
7     def __init__(self, name, age, marks):
8         """
9         Initialize a Student instance.
10
11         Parameters:
12         name (str): The student's name.
13         age (int): The student's age.
14         marks (list of int): List of marks in different subjects.
15         """
16         self.name = name
17         self.age = age
18         self.marks = marks
19
20     Tabnine | Edit | Test | Explain | Document | Qodo Gen: Options | Test this method
21     def details(self):
22         """Print the student's name and age in a readable format."""
23         print(f"Name: {self.name}, Age: {self.age}")
24
25     Tabnine | Edit | Test | Explain | Document | Qodo Gen: Options | Test this method
26     def total(self):
27         """Return the total marks obtained by the student."""
28         return sum(self.marks)
29
30 # Example usage:
31 student = Student("Alice", 20, [85, 90, 78])
32 student.details()
33 print("Total Marks:", student.total())
34
```

Output:

```
PS C:\Users\HP\Desktop\AI-ASSISTED CODING> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/HP/Desktop/AI-ASSISTED CODING/13.3.task3.py"
Name: Alice, Age: 20
Total Marks: 253
PS C:\Users\HP\Desktop\AI-ASSISTED CODING> |
```

Observation:

1. Variable names changed to **name**, **age**, and **marks** for clarity.
2. Marks are stored in a **list**, enabling flexible subject count.
3. `total()` uses Python's built-in `sum()` for cleaner and scalable calculation.
4. Added **docstrings** explaining class purpose and method behavior.
5. The `details()` method uses an **f-string** for more readable and concise output.
6. Class design is **modular** and **easy to extend** with additional features.
7. Cleaner constructor supports any number of marks, improving **scalability**.
8. Overall, the class is more **Pythonic**, maintainable, and user-friendly

Task Description -#4 Inefficient Loop Factoring

Task: Refactor this inefficient loop with AI help

Python Code

```
nums = [1,2,3,4,5,6,7,8,9,10]
```

```
squares = []
```

```
for i in nums:
```

```
    squares.append(i * i)
```

Prompt:

Refactor the following inefficient loop in Python to make it more concise and efficient.

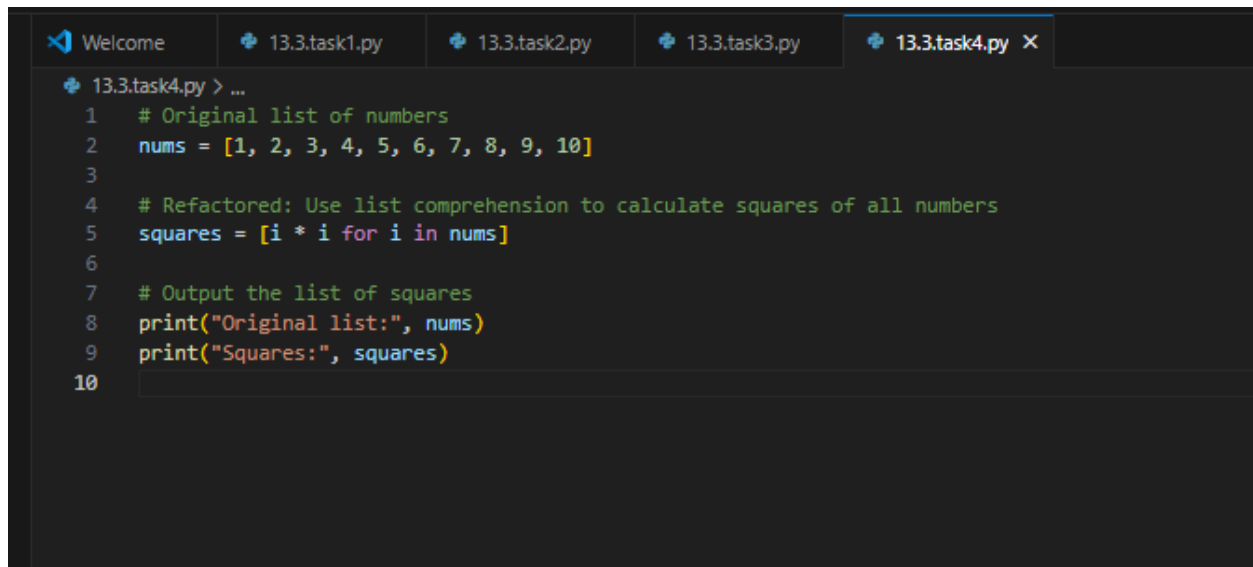
Code:

```
nums = [1,2,3,4,5,6,7,8,9,10]
squares = []
for i in nums:
    squares.append(i * i)
```

Requirements:

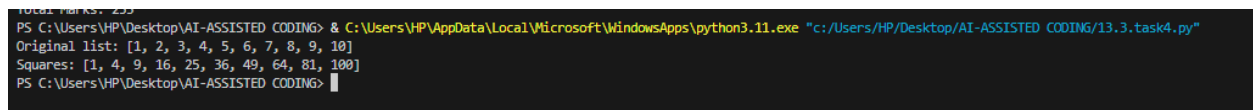
- Use **list comprehension** to replace the loop.
- The refactored code should be **clean, readable, and efficient**.
- Preserve the same functionality (list of squares of numbers in `nums`).

CODE GENERATED:



```
Welcome 13.3.task1.py 13.3.task2.py 13.3.task3.py 13.3.task4.py X
13.3.task4.py > ...
1 # Original list of numbers
2 nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
3
4 # Refactored: Use list comprehension to calculate squares of all numbers
5 squares = [i * i for i in nums]
6
7 # Output the list of squares
8 print("Original list:", nums)
9 print("Squares:", squares)
10
```

Output:



```
Total Marks: 255
PS C:\Users\HP\Desktop\AI-ASSISTED CODING> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/HP/Desktop/AI-ASSISTED CODING/13.3.task4.py"
Original list: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Squares: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
PS C:\Users\HP\Desktop\AI-ASSISTED CODING> |
```

Observation:

1. **Conciseness:** The entire operation to create the list of squares is now a single line instead of multiple lines with an explicit loop and list append calls.
2. **Readability:** List comprehensions express the intention clearly — "generate a list of squares for each element in `nums`" — making it easier to understand at a glance.
3. **Efficiency:** List comprehensions in Python are internally optimized and often faster than manually appending elements within a loop.

4. **Pythonic Style:** Using list comprehensions is considered a best practice and idiomatic Python, aligning with the language's design philosophy.
5. **Maintainability:** Less code means fewer chances for bugs or mistakes. The refactored code is easier to maintain and extend if needed.
6. **Performance:** Especially for larger lists, list comprehensions avoid the overhead of multiple `append` method calls.
7. **Functional Programming Paradigm:** List comprehensions fit well with functional programming styles, encouraging declarative rather than imperative coding.
8. **Output Validation:** The refactored code includes print statements that help verify the correctness of the operation by showing both the original list and the resulting squares.